DSA5208: Scalable Distributed Computing for Data Science

Lecture 4: Introduction to Apache Spark

Zhenning Cai

Department of Mathematics, National University of Singapore

AY2026/2027, Semester 1

1 Introduction to Apache Spark

2 Structured APIs for DataFrames

3 Apache Spark with GPU acceleration

4 Summary

What is Apache Spark?

Apache Spark is a unified engine for large-scale data analytics.



- **Unified:** Spark supports a wide range of analytics tasks from simple data loading and SQL queries to machine learning and streaming computation, with a consistent set of APIs.
- **Data analytics:** The scope of Spark is limited to an analytics engine. Unlike Hadoop, Spark does not provide a storage system. But Spark runs well on many storage systems including HDFS.
- **Large-scale:** Spark is built for large and parallel computations on clusters of as many as thousands of nodes. The algorithms are highly scalable and efficient.

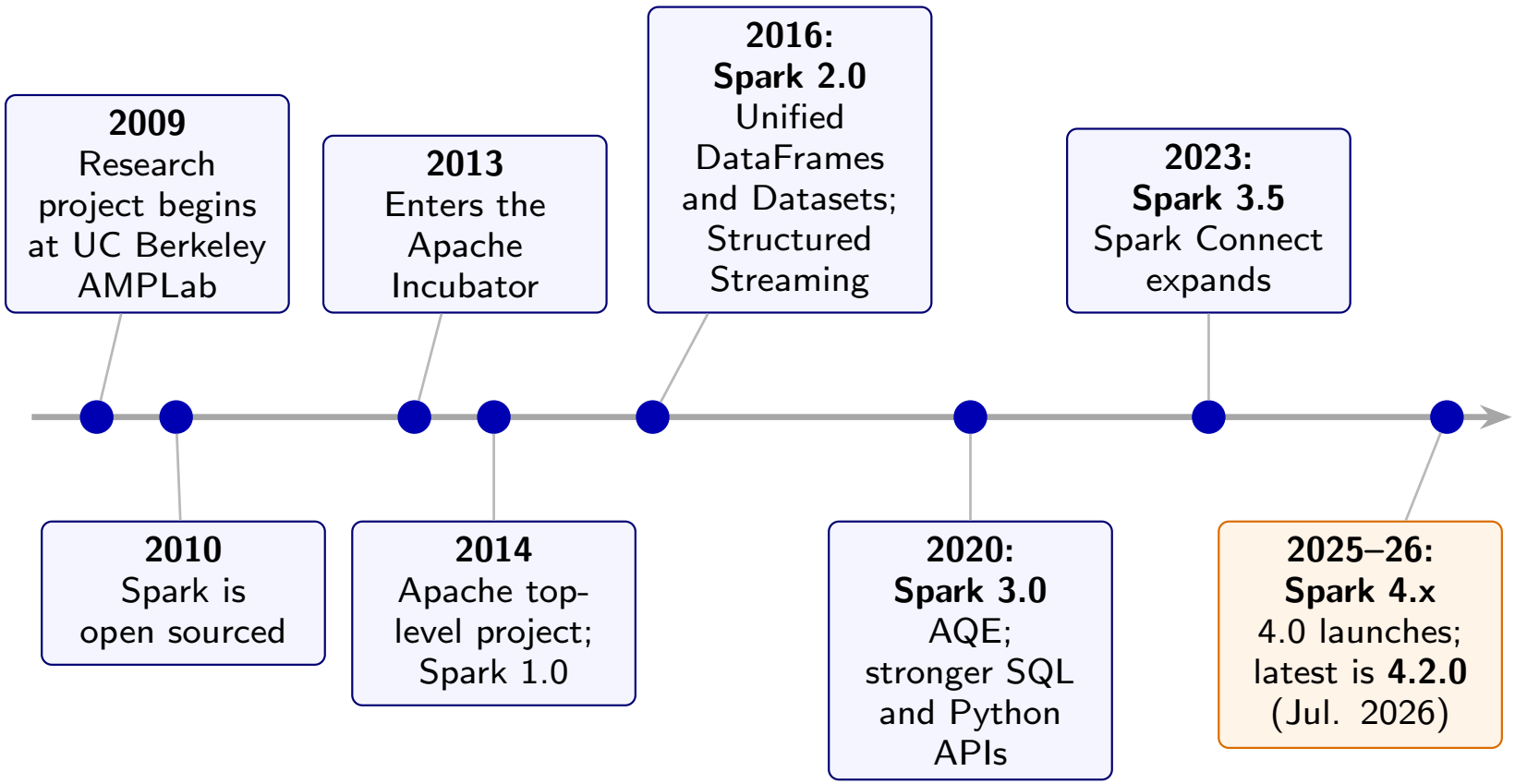
Apache spark is a multi-language engine for data engineering. It supports Python, SQL, Scala, Java and R.

Components of Spark

Apache Spark supports a rich set of high-level tools, including

- Spark SQL for SQL and structured data processing.
- Pandas API on Spark for pandas workloads.
- MLlib for machine learning.
- GraphX for graph processing.
- Structured streaming for incremental computation and stream processing.

History of Apache Spark



Sources: Apache Spark project history and release notes; updated August 2026.

`https://spark.apache.org/news/` `https://spark.apache.org/releases/`

First example with PySpark

Dataset: The dataset records the on-time performance of US domestic flights operated by 14 air carriers:

YEAR	MONTH	DAY	DAY_OF_WEEK	AIRLINE	FLIGHT_NUMBER	···	SCHEDULED_ARRIVAL	ARRIVAL_TIME	ARRIVAL_DELAY	···
2015	1	1	4	AS	98	···	0430	0408	-22	···
2015	1	1	4	AA	2336	···	0750	0741	-9	···
2015	1	1	4	US	840	···	0806	0811	5	···
···	···	···	···	···	···	···	···	···	···	···
2015	12	31	4	B6	1503	···	0440	0432	-8	···
2015	12	31	4	B6	333	···	0340	0330	-10	···
2015	12	31	4	B6	839	···	0440	0442	2	···

Source: `https://www.kaggle.com/datasets/usdot/flight-delays/data`

- The dataset contains 5,819,079 rows and 31 columns.
- The size of the csv file is about 565M.
- Some information is missing in the dataset.

Which carrier has the least average delay? How to sort the data in descending order by the arrival delay?

First example with PySpark

- Launch the Python console:

```
$ pyspark
```

- Load the dataframe from a csv file:

```
>>> df = spark.read.csv("flights.csv",
...                     inferSchema=True, header=True)
```

- Find the average arrival delay for each carrier:

```
>>> from pyspark.sql.functions import avg
>>> df.groupby("AIRLINE").agg(
...     avg("ARRIVAL_DELAY").alias("AVERAGE_DELAY")).show()
```

- Write the sorted results into files:

```
>>> df.orderBy("ARRIVAL_DELAY", ascending=False).write.csv(
...     "ordered_flights.csv", header=True)
```

1 Introduction to Apache Spark

2 Structured APIs for DataFrames

3 Apache Spark with GPU acceleration

4 Summary

1 Introduction to Apache Spark

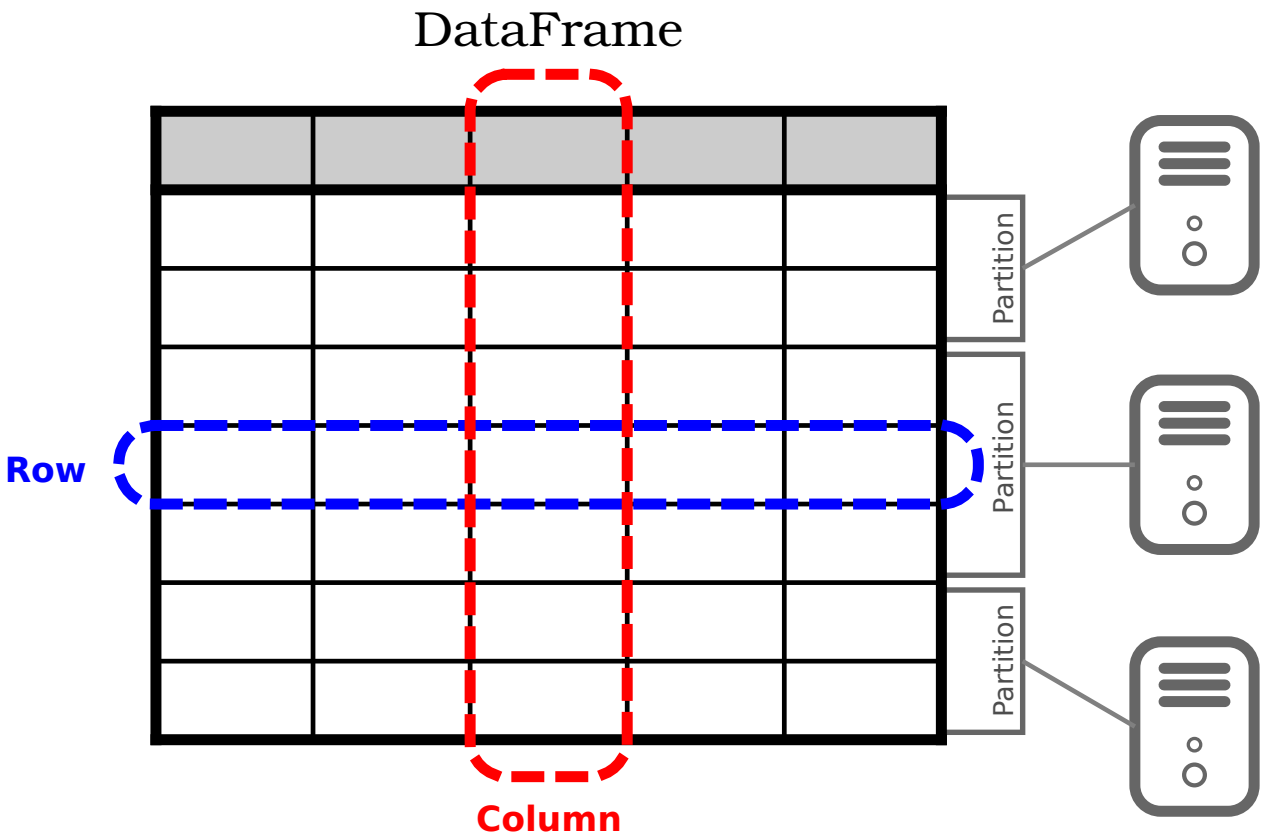
2 Structured APIs for DataFrames
- DataFrames and some APIs
- Execution of structured APIs

3 Apache Spark with GPU acceleration

4 Summary

DataFrames

- A DataFrame represents a table of data with rows and columns.
- Spark can break up a DataFrame into partitions, which may be stored in different computer nodes.
- Each partition contains a collection of rows of the DataFrame.



Schemas, columns and rows

- A schema defines the column names and types of a DataFrame.

```
>>> df = spark.read.csv("flights.csv", inferSchema=True, header=True)
>>> df.printSchema()
>>> schema = df.schema
>>> from pyspark.sql.types import StringType
>>> schema.fields[10].dataType = schema.fields[11].dataType = \
...     schema.fields[20].dataType = schema.fields[21].dataType = StringType()
>>> df = spark.read.csv("flights.csv", schema, header=True)
```

- Each row in a DataFrame is a single record.

```
>>> df.first() # print the first row
>>> df.take(3) # print the first three rows
```

We can create rows and use them to make a DataFrame:

```
>>> from pyspark.sql import Row
>>> rows = [Row("Hello", 1, True), Row("World", 2, False)]
>>> myDf = spark.createDataFrame(rows, ["String", "Number", "Boolean"])
>>> myDf.show()
```

Schemas, columns and rows

- A schema defines the column names and types of a DataFrame.

```
>>> df = spark.read.csv("flights.csv", inferSchema=True, header=True)
>>> df.printSchema()
>>> schema = df.schema
>>> from pyspark.sql.types import StringType
>>> schema.fields[10].dataType = schema.fields[11].dataType = \
...     schema.fields[20].dataType = schema.fields[21].dataType = StringType()
>>> df = spark.read.csv("flights.csv", schema, header=True)
```

- Columns are logical constructions that cannot exist outside the context of a DataFrame.

```
>>> df.columns
>>> df.YEAR
>>> df.YEAR - 2000
```

Select, add and remove columns

- **Select columns:** There are multiple ways to specify columns.

```
>>> from pyspark.sql.functions import col, expr
>>> arrInfo = df.select("YEAR", df.MONTH, col("DAY"),
...     expr("AIRLINE"), df.colRegex("`.*ARRIVAL.*`"))
>>> arrInfo.show(3)
```

- **Add columns:** New columns can be added by using either `select` or `withColumn`:

```
>>> arrInfo.select("*", (col("ARRIVAL_DELAY") < 0).alias("ON TIME")).show(3)
>>> arrInfo.withColumn("ON TIME", col("ARRIVAL_DELAY") < 0).show(3)
```

- **Rename columns:**

```
>>> arrInfo.withColumnRenamed("ARRIVAL_DELAY", "ARR_DELAY").show(3)
```

- **Remove columns:**

```
>>> arrInfo.drop("YEAR").show(3)
```

Filter, sample, add rows

- **Filter rows:** Both `filter` and `where` functions can pick rows satisfying certain conditions:

```
>>> arrInfo.filter(col("ARRIVAL_DELAY") > 1440).show()
>>> arrInfo.where(col("AIRLINE") == "VX").\
...     where(col("ARRIVAL_DELAY") < -60).show()
```

- **Sample rows:**

```
>>> seed = 5
>>> withReplacement = False
>>> fraction = 2e-6
>>> arrInfo.sample(withReplacement, fraction, seed).show()
```

- **Add rows:** To add rows, one needs to create a new DataFrame and take the union of two DataFrames.

```
>>> newRows = [Row(2016, 1, 1, "AA", "1327", "1340", 13),
...            Row(2016, 1, 1, "AA", "1724", "1756", 32)]
>>> newDf = spark.createDataFrame(newRows, arrInfo.schema)
>>> newArrInfo = arrInfo.union(newDf)
>>> spark.createDataFrame(newArrInfo.tail(5), arrInfo.schema).show()
```

Sort and limit

- **Sort:** Functions `sort` and `orderBy` are responsible for sorting rows.

```
>>> from pyspark.sql.functions import isnull
>>> newArrInfoNullRemoved = newArrInfo.filter(~isnull("ARRIVAL_DELAY")) \
...                                                  # Remove missing values
>>> newArrInfoNullRemoved.sort("ARRIVAL_DELAY").show(5)
>>> newArrInfoNullRemoved.orderBy(col("ARRIVAL_DELAY").desc()).show(20)
>>> newArrInfoNullRemoved.orderBy("AIRLINE", \
...     col("ARRIVAL_DELAY").desc()).show(20)
```

Use `asc_nulls_first`, `asc_nulls_last`, `desc_nulls_first`, `desc_nulls_first` to specify where the missing values should appear in an ordered DataFrame.

```
>>> newArrInfo.sort(col("ARRIVAL_DELAY").desc_nulls_first(),
...     "AIRLINE").show(5)
>>> newArrInfo.sort(col("ARRIVAL_DELAY").desc_nulls_last(),
...     "AIRLINE").show(5)
```

- **Limit:** Function `limit` takes the first few rows of a DataFrame.

```
>>> newArrInfoNullRemoved.limit(6).show()
```

Aggregation functions

- Aggregation functions like `count` and `avg`

```
>>> import pyspark.sql.functions as psf
>>> df.select(
...     psf.countDistinct("YEAR", "MONTH", "DAY").alias("NUM_DAYS"),
...     psf.countDistinct("AIRLINE").alias("NUM_AIRLINES"),
...     psf.count("DEPARTURE_TIME").alias("NUM_DEPARTED_FLIGHTS"),
...     psf.max("ARRIVAL_DELAY").alias("MAX_ARR_DELAY"),
...     psf.avg("ARRIVAL_DELAY").alias("AVG_ARR_DELAY")
... ).show()
```

Other aggregation functions include `first`, `last`, `sum`, `sumDistinct`, `variance`, `stddev`, `cov`, ...

- We can group the rows according to a certain column before applying aggregation functions. Then the aggregation functions will be applied within each group of data.

```
>>> df.groupBy("AIRLINE").agg(
...     psf.max("ARRIVAL_DELAY").alias("MAX_ARR_DELAY"),
...     psf.min("ARRIVAL_DELAY").alias("MIN_ARR_DELAY"),
...     psf.avg("ARRIVAL_DELAY").alias("AVG_ARR_DELAY")
... ).sort(col("AVG_ARR_DELAY").desc()).show()
```

1 Introduction to Apache Spark

2 Structured APIs for DataFrames
- DataFrames and some APIs
- Execution of structured APIs

3 Apache Spark with GPU acceleration

4 Summary

Lazy evaluation

Lazy evaluation means that Spark will wait until the very last moment to execute computational instructions.

- The following operations do not even read the file, but they build up a plan for the computations.

```
>>> df = spark.read.csv("flights.csv", schema)
>>> dfNullRemoved = df.filter(~psf.isnull("ARRIVAL_DELAY"))
>>> dfSorted = dfNullRemoved.select(
...     "YEAR", "MONTH", "DAY", "AIRLINE", df.colRegex("`.*ARRIVAL.*`")
... ).sort(col("ARRIVAL_DELAY").desc())
```

- To view the plan, use

```
>>> dfSorted.explain(mode = "formatted")
```

- The following commands will carry out computations:

```
>>> dfNullRemoved.count()
>>> dfSorted.first()
```

Transformations

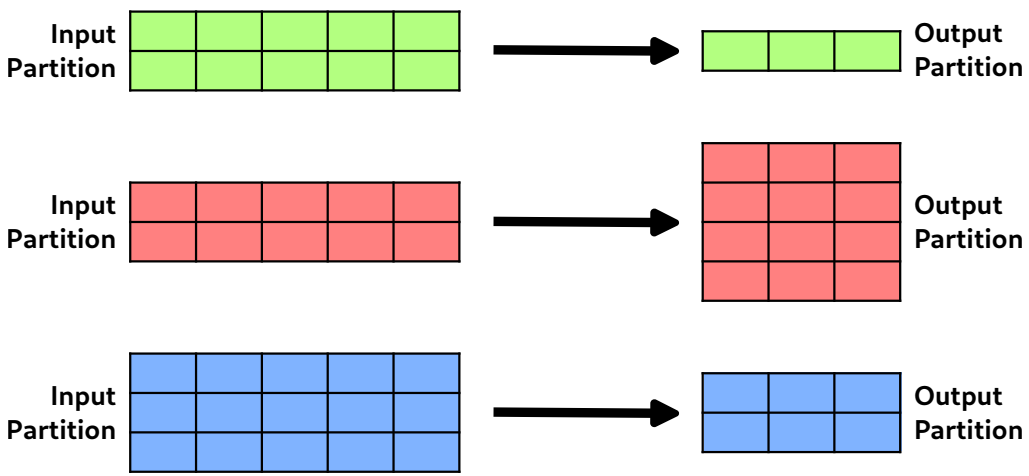
Transformations produce new DataFrames from existing DataFrames. They are lazy operations that do not generate data.

Transformations

Transformations produce new DataFrames from existing DataFrames. They are lazy operations that do not generate data.

Two types of transformations:

- **Narrow transformations:** Transformations in which each input partition contributes to at most one output partition.



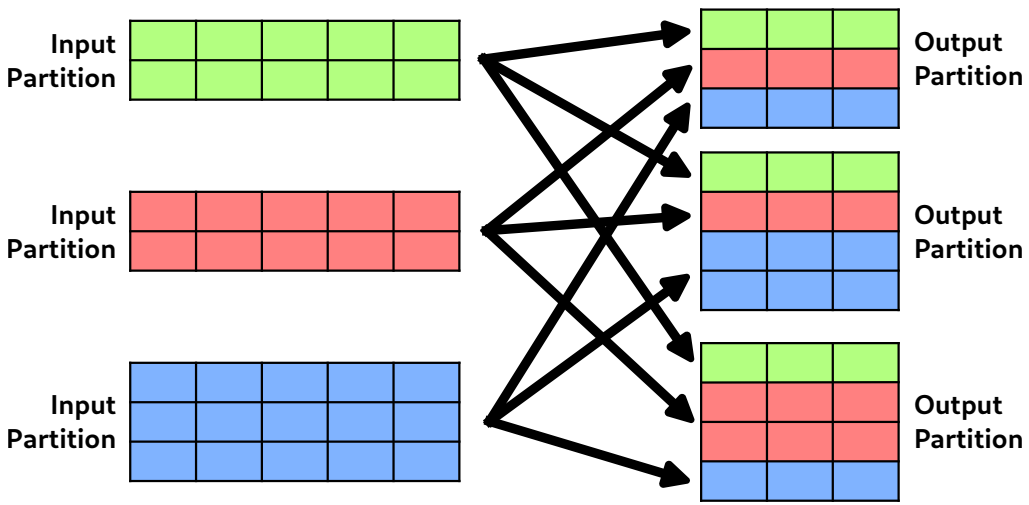
Examples: `filter`, `withColumn`.

Transformations

Transformations produce new DataFrames from existing DataFrames. They are lazy operations that do not generate data.

Two types of transformations:

- **Wide transformations:** Transformations in which each input partition may contribute to more than one output partition.



Examples: `sort`, `select` with aggregation functions.

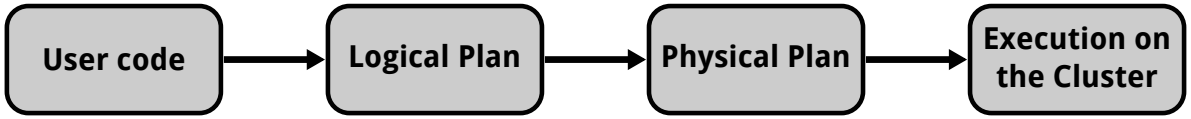
Actions

An action instructs Spark to compute a result from a series of transformations.

Three types of actions:

- Actions to view data in the console.
  Example: `count`.
- Actions to collect data to native objects.
  Example: `collect`.
- Actions to write to output data sources.
  Example: `saveAsTextFile`.

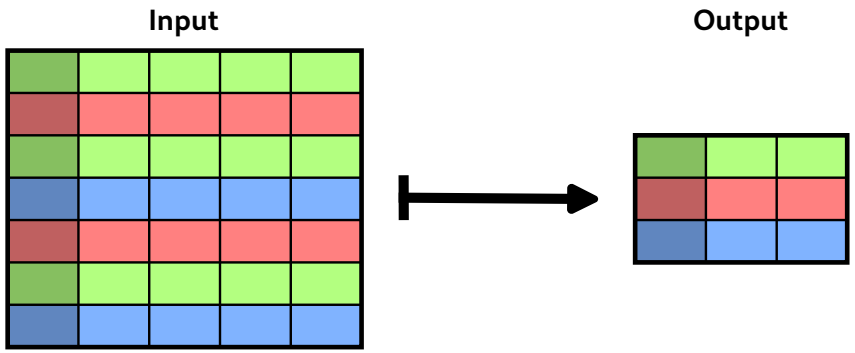
Logical and physical plans



- **Logical plan:** Logical plans refer to a set of transformations that do not refer to actual functions to call.
- **Physical plan:** Physical plans specify how the logical plan will execute on the cluster, including which function will be called.

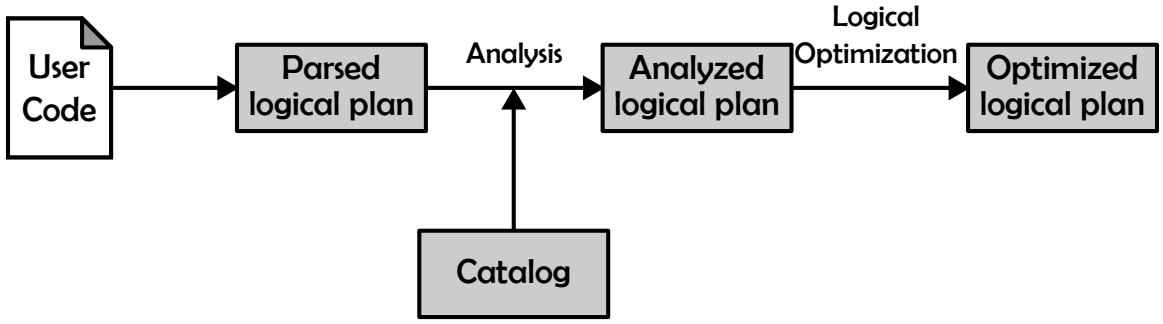
Example: Consider a `sum` or `avg` aggregation with `groupBy`.

- **Logical plan:**



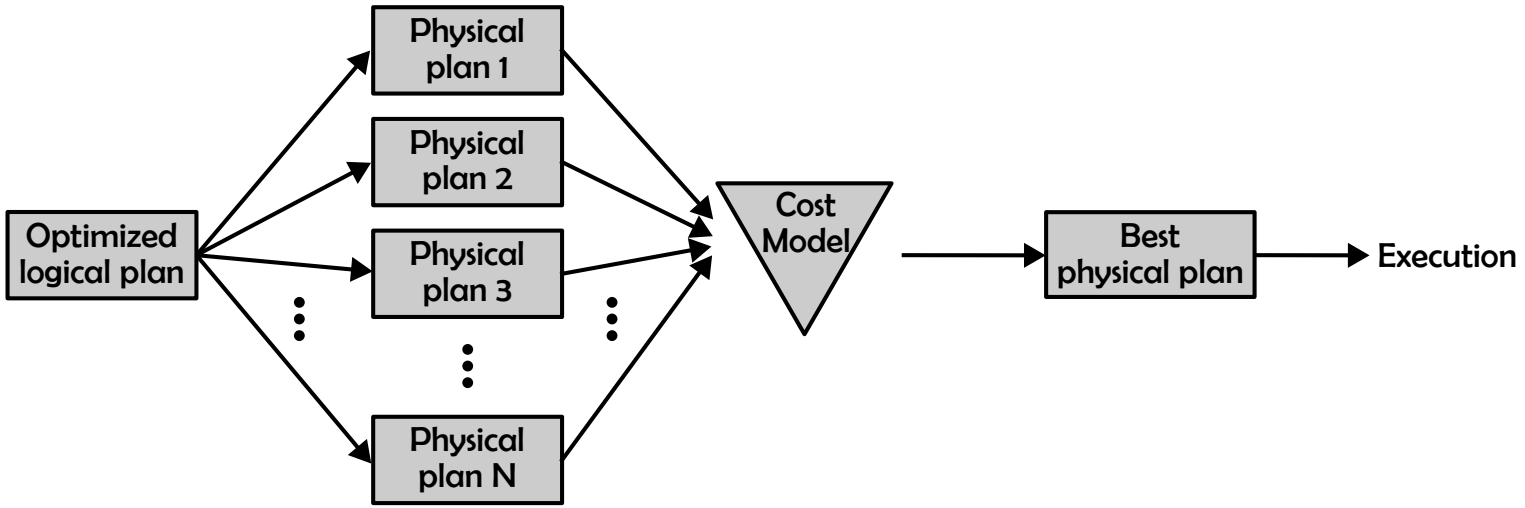
- **Physical plan:** Hash-based aggregation or Sort-based aggregation?

Logical planning



- The parsed logical plan is first generated purely according to the user code. It builds a tree structure without checking anything related to the actual data, such as whether the columns specified exist and whether the data types match.
- The analyzed logical plan resolves the parsed plan by expanding regular expressions, checking column names, verifying data types, etc. It requires the Catalog, which is a repository of Spark tables, DataFrames, and DataSets.
- The analyzed plan is passed on to a Catalyst optimizer, which performs possible optimizations such as combining filters and reordering operations.

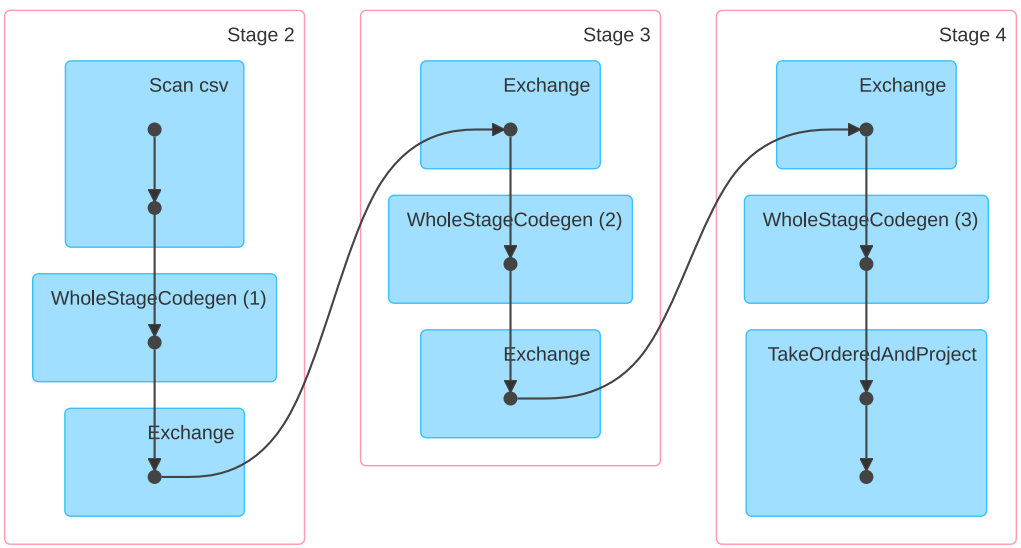
Physical planning



- Spark generates several physical plans according to the optimized logical plan.
- The physical plans are sent to a cost model for evaluation. Then the best plan is picked according to the cost model.
- The best physical plan is sent to the computing nodes for execution.
- **Warning:** The best physical plan may not be the final plan if AQE (Adaptive Query Execution) is enabled.

Jobs, stages and tasks

- A Spark job is created when an action is executed[1].
- A job contains several stages, each including some tasks.
- A wide transformation brings one stage to the next.
- A task represents a single operation that can be executed on a partitioned subset of data. Tasks are executed by the worker nodes in parallel.



[1] Multiple jobs may be created when AQE is enabled.

1 Introduction to Apache Spark

2 Structured APIs for DataFrames

3 Apache Spark with GPU acceleration

4 Summary

RAPIDS/CUDA-X Accelerator for Apache Spark

RAPIDS

`https://developer.nvidia.com/topics/ai/data-science/cuda-x-for-data-science`:

> *CUDA-X™ is a collection of highly optimized, domain-specific libraries built on CUDA™ that includes a suite of open source libraries for accelerated data science.*

`https://www.nvidia.com/en-us/technologies/cuda-x/`

> **Zero-Code Acceleration:** *Speeds up existing tabular data and machine learning workflows in pandas, Polars, scikit-learn, and Apache Spark without code changes.*

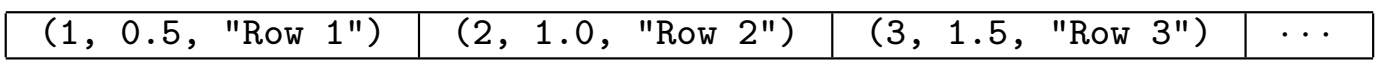
Row-based and column-based storages

A key difference between Spark and CUDA-X DataFrames is the underlying data layout: Spark commonly uses a row-based layout internally, while CUDA-X uses a column-based layout.

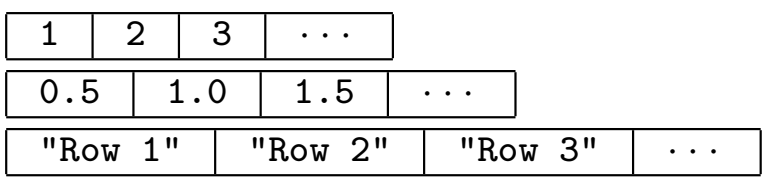
A sample DataFrame:

Integer	Real number	String
1	0.5	"Row 1"
2	1.0	"Row 2"
3	1.5	"Row 3"
⋮	⋮	⋮

- Row-based storage:



- Column-based storage:



Why column-based storage?

The column-based storage allows thousands of GPU threads to access adjacent values and apply the same operation to entire vectors, enabling high throughput.

[2] Test performed on a laptop with Intel i7-13800H CPU.

Why column-based storage?

The column-based storage allows thousands of GPU threads to access adjacent values and apply the same operation to entire vectors, enabling high throughput.

Even on CPUs, column-based storage is often more efficient for aggregation operations due to better cache locality.

[2] Test performed on a laptop with Intel i7-13800H CPU.

Why column-based storage?

The column-based storage allows thousands of GPU threads to access adjacent values and apply the same operation to entire vectors, enabling high throughput.

Even on CPUs, column-based storage is often more efficient for aggregation operations due to better cache locality.

Toy example: Add up the column "`Real number`".[2]

Integer	Real number	String
1	0.5	"Row 1"
2	1.0	"Row 2"
3	1.5	"Row 3"
⋮	⋮	⋮

Test with 10,000,000 rows:

- Row-based storage: 0.040624 s
- Column-based storage: 0.006117 s

[2] Test performed on a laptop with Intel i7-13800H CPU.

`filter()` operation in Spark

The `filter` method of a DataFrame returns a new DataFrame containing only the rows that satisfy a given predicate.

[3] An `UnsafeRow` is a low-level representation of a row that allows for efficient memory access.

`filter()` operation in Spark

The `filter` method of a DataFrame returns a new DataFrame containing only the rows that satisfy a given predicate.

Implementation:

- Retrieve a row in the partition.
- Evaluate the predicate on the row.
- If the predicate holds, create an `UnsafeRow`[3] that is identical to the row being checked.
- Append the `UnsafeRow`.

[3] An `UnsafeRow` is a low-level representation of a row that allows for efficient memory access.

`filter()` operation in Spark

The `filter` method of a DataFrame returns a new DataFrame containing only the rows that satisfy a given predicate.

Implementation:

- Retrieve a row in the partition.
- Evaluate the predicate on the row.
- If the predicate holds, create an `UnsafeRow`[3] that is identical to the row being checked.
- Append the `UnsafeRow`.

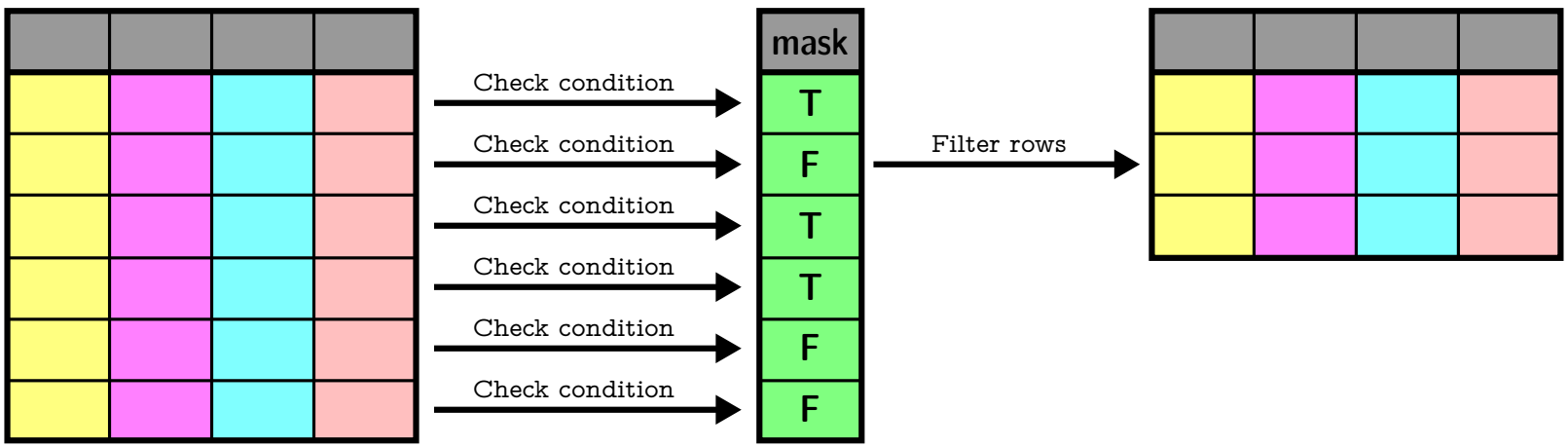
☞ The procedure is wrapped into a *WholeStageCodegen* operator, which can be found via `DataFrame.explain(mode = "codegen")`.

☞ The rows are copied because the input row may not be an `UnsafeRow`, and the generated code may contain other operations that modify the number of fields.

[3] An `UnsafeRow` is a low-level representation of a row that allows for efficient memory access.

`filter()` operation in CUDA-X

CUDA-X performs the `filter` operation on the GPU batch by batch. For each batch, the conditions are checked in parallel by thousands of GPU threads.



`mask: [1, 0, 1, 1, 0, 0]` —Exclusive Scan→ `pos: [0, 1, 1, 2, 3, 3]`

```
if (mask[i])
    send row i to location pos[i]
```

`filter()` operation in CUDA-X

- CUDA-X does not generate codes for any operations.
- If the predicate contains operations not supported by cuDF, the filter falls back to the Spark version. In this case, the DataFrame must be converted via `ColumnarToRow` before being filtered.
- Filtering may generate many small batches. By default, CUDA-X will try to coalesce batches to avoid too many fragments.

Projecting DataFrames

Projecting a DataFrame refers to the act of selecting a subset of columns from the DataFrame or creating new columns based on existing ones. Examples include

- Selecting/Reordering specific columns:

```
df.select("col2", "col1")
```

- Creating new columns:

```
df.withColumn("newCol", col("col1") + col("col2"))
```

- Renaming columns:

```
df.withColumnRenamed("oldName", "newName")
```

Projecting DataFrames

Projecting a DataFrame refers to the act of selecting a subset of columns from the DataFrame or creating new columns based on existing ones. Examples include

- Selecting/Reordering specific columns:

```
df.select("col2", "col1")
```

- Creating new columns:

```
df.withColumn("newCol", col("col1") + col("col2"))
```

- Renaming columns:

```
df.withColumnRenamed("oldName", "newName")
```

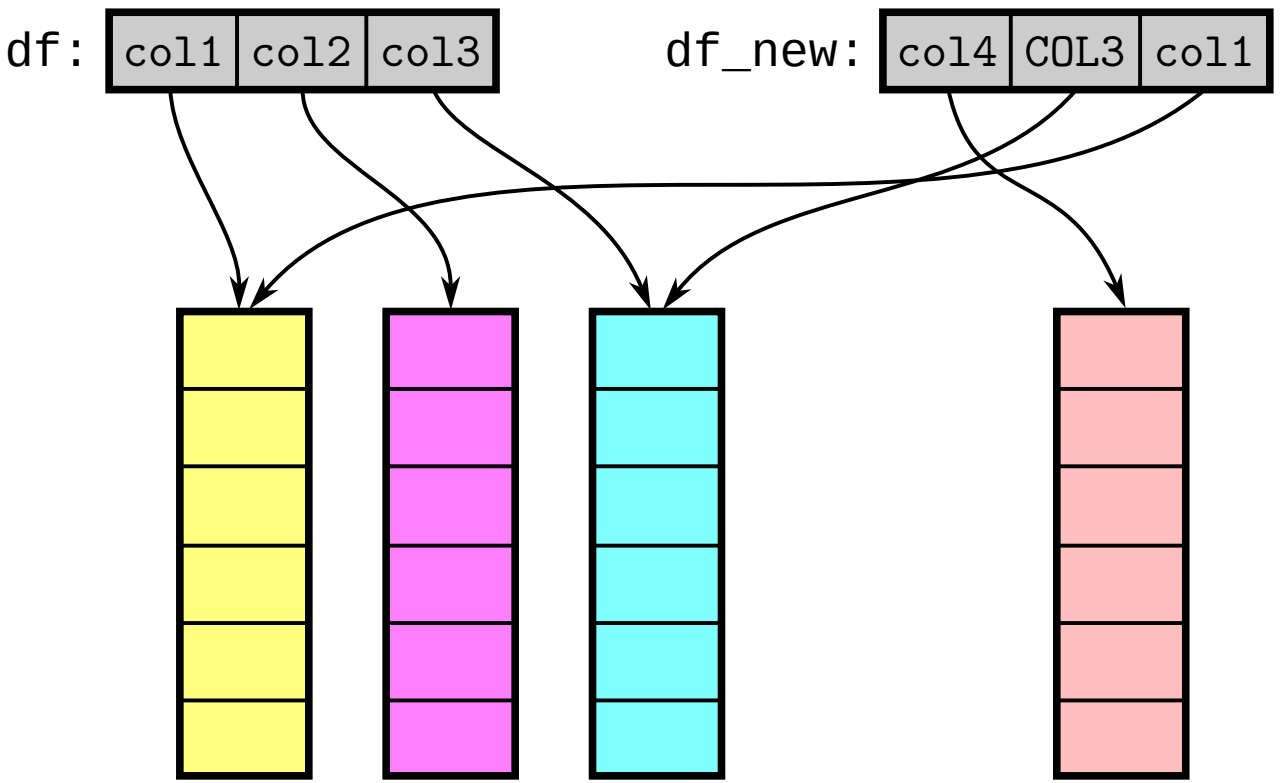
Spark's implementation:

- The new schema is generated during planning.
- For each row in the partition, a new `UnsafeRow` is created by selecting/reordering fields or evaluating expressions.

Project in CUDA-X

In CUDA-X, due to the column-based storage, it is unnecessary to create new rows if the projection only involves selecting/reordering columns.

```
df_new = df.select((df.col2 + df.col3).alias("col4"),
                   df.col3.alias("COL3"), df.col1)
```



1 Introduction to Apache Spark

2 Structured APIs for DataFrames

3 Apache Spark with GPU acceleration

4 Summary

Summary

- Basic operations of DataFrames
  - Manipulations of rows and columns
  - Sort
  - Aggregations
- Lazy evaluation
  - Wide and narrow transformations
  - Logical and physical plans
- GPU acceleration with CUDA-X
  - Data layout: row-based vs column-based
  - Implementation of filters and projections